

Interfaze стал в 10 раз дешевле благодаря более эффективному использованию токенов и кэшированию

[copy markdown](#)

Тот же запрос на распознавание текста, который на прошлой неделе стоил 0,041 доллара, теперь стоит 0,0038 доллара, и вам не нужно менять ни строки кода.

Мы добились этого благодаря двум факторам:

- Эффективность модели позволяет генерировать и подсчитывать меньше токенов.
- Токены для схожих или повторяющихся задач не кэшируются и не оплачиваются повторно.

Что изменилось

Interfaze – это сочетание специализированных архитектур, объединенных в единую модель. Поскольку не все модели одинаковы, созданные токены не всегда используются полностью, особенно при наличии дублирования в разных задачах.

Мы потратили время на стандартизацию каждого результата, удалили неиспользуемые токены и убедились, что учитываем только то, что было использовано для получения результата.

Хотя цена за миллион токенов осталась прежней, каждый запрос теперь стоит дешевле, поскольку мы выпускаем меньше токенов и учитываем меньшее их количество.

Эффективность токенов в режиме выполнения задачи

Режим выполнения задачи запускает одну встроенную функцию без активации полной модели. Именно на этом этапе достигается большая часть экономии.

Каждая строка ниже – это один и тот же запрос, измеренный до и после внесения изменений.

Задача	Токены до	Токены сейчас	Стоимость до	Cost now	Cheaper
ocr	12,568	1,816	\$0.04142	\$0.00381	10.9x
gui_detection	5,231	1,294	\$0.01580	\$0.00208	7.6x
forecast	10,829	3,178	\$0.03471	\$0.00808	4.3x
web_search	8,294	3,107	\$0.02650	\$0.00837	3.2x

Задача	Токены до	Токены сейчас	Стоимость до	Cost now	Cheaper
object_detection	5,182	2,705	\$0.01572	\$0.00710	2.2x
speech_to_text	2,868	2,256	\$0.00740	\$0.00528	1.4x

Примеры: одностраничное изображение счета за ocr , полный скриншот Википедии за gui_detection , ежедневная серия из 12 пунктов за forecast , один поисковый запрос за web_search , фотография строительной площадки за object_detection и 52-секундный WAV-файл за speech_to_text .

Эффективность токенов при запросах к полной модели

При запуске полной модели, позволяющей использовать несколько возможностей одновременно, модель может генерировать больше токенов и становится гораздо более гибкой в плане использования.

Задача	Токены до	Токены сейчас	Стоимость до	Cost now	Cheaper
ocr	26,014	15,262	\$0.06497	\$0.02736	2.4x
forecast	14,911	7,244	\$0.04134	\$0.01465	2.8x
object_detection	9,697	7,159	\$0.02281	\$0.01398	1.6x
gui_detection	24,254	20,335	\$0.05309	\$0.03943	1.3x
speech_to_text	8,581	7,702	\$0.01662	\$0.01357	1.2x
web_search	64,408	59,221	\$0.11317	\$0.09503	1.2x
Все шесть	147,865	116,923	\$0.31200	\$0.20402	1.5x

web_search и gui_detection дают наилучшие результаты, поэтому их общие показатели в диалоговом режиме остаются высокими. Результат возвращается в модель, и от оплаты этих токенов никуда не деться.

Если вам нужен только необработанный результат, используйте режим выполнения задачи и полностью пропустите второй проход.

Как это выглядит в динамике

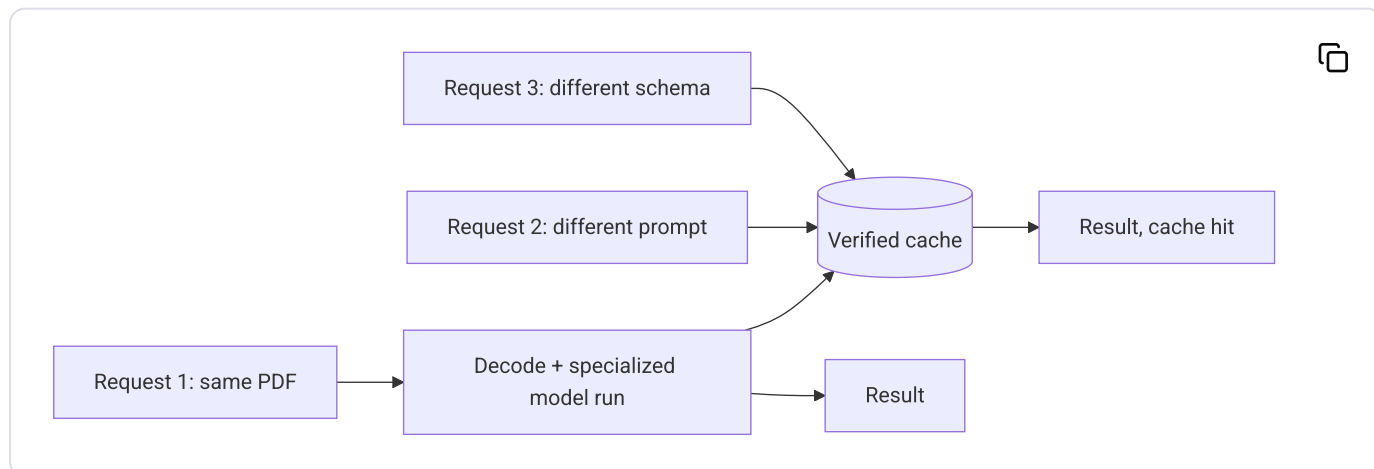
Нагрузка	До	Сейчас
10 000 страниц счетов, ocr	\$414.20	\$38.10
10 000 web_search запросов	\$265.00	\$83.70
10 000 forecast запросов	\$347.10	\$80.80

предостережение о том, как все это следует воспринимать. Каждая цифра – это один пример для каждой возможности, измеренный при первом вызове, когда кэш пуст, поэтому рассматривайте эти показатели как репрезентативные, а не как точные средние значения.

Кэширование для повторной работы с одним и тем же файлом

Большинство операций с документами – это не один запрос на файл. Вы извлекаете поля, затем выполняете операцию повторно с другой схемой, затем задаете дополнительный вопрос – и все это с одним и тем же PDF-файлом.

Теперь Interfaze кэширует дорогостоящую часть первого прохода. При повторных запросах к одному и тому же файлу или очень похожей подсказке данные используются повторно, а не декодируются заново и не запускаются специализированная модель.



Running multiple requests against the same file is roughly 10x cheaper this way. Caching is on by default and cached outputs are not charged again, so there is no flag to set and no separate cache bill.

typescript ▾

Interfaze SDK OpenAI SDK

```
import { Interfaze } from "interfaze";

const interfaze = new Interfaze();

const paper = "https://arxiv.org/pdf/2602.04101";

const questions = [
  "List the authors of this paper.",
  "Summarize the methodology in three bullet points.",
  "Extract every table caption.",
];
```

```
for (const question of questions) {  
  const res = await interfaze.chat.completions.create({
```

One thing worth knowing: cache hits depend on the file being addressable the same way each time. Pass a stable URL rather than re-uploading the same bytes as base64 on every call and you will hit the cache far more often.

Getting to the cheapest path

The changes above are automatic, but a few habits compound on top of them.

1. **Pass files as URLs, not base64.** Faster, cheaper, and it makes the cache work for you. See [handling files](#).
2. **Batch several capabilities into one request.** Transcribe, translate, and classify in a single call and read the raw outputs from [precontext](#) instead of paying for three round trips.
3. **Use structured outputs instead of describing a schema in the prompt.** A JSON Schema pasted into a prompt makes the model retry until it gets the shape right, and you pay for every retry.
4. **Use run-task mode when you're only interested in raw output.** Raw fixed structured data output, up to 10.9x on OCR.

More patterns in [lowering costs and improving performance](#).

Takeaway

- **Run-task requests** got 1.4x to 10.9x cheaper, with OCR seeing the largest drop.
- **Full model requests** got 1.2x to 2.8x cheaper, since the model still has to read the tool result.
- **Repeated requests on one file** are roughly 10x cheaper with caching, which is on by default.
- **Per token pricing did not change.** You are billed fewer tokens for identical work.

Nothing here needs a migration, an upgrade, or a new parameter. Your existing calls just got cheaper.

- Start building in the [dashboard](#)
- Read the [run-task docs](#)
- Join the [Discord](#)

PRODUCT	CAPABILITIES
Playground	Vision
OCR	Web
Models	Audio
Leaderboards	GUI
Pricing	Compute
DEVELOPERS	RESOURCES
Docs	Blog
API Ref	FAQs
Structured Outputs	Help
Integration	
	LEGAL
COMPANY	Privacy
Careers	Terms
Sales	DPA
Status	

